BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Lê Minh Hiếu
MSSV / Lớp	3124720023 / Trí Tuệ Nhân Tạo

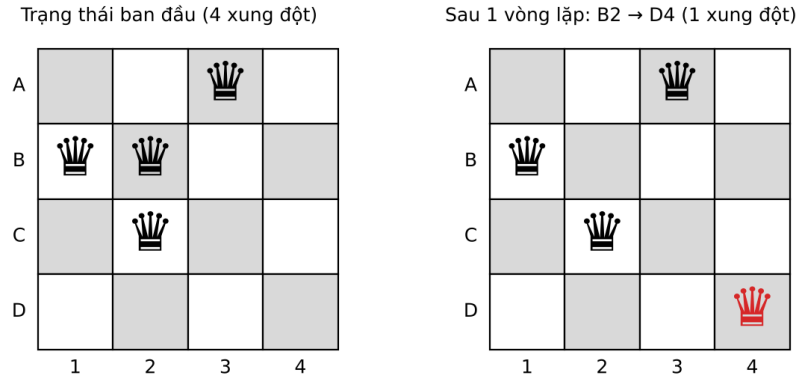
Quy ước: ô bàn cờ ghi theo (hàng A–D)(cột 1–4), ví dụ A3 là hàng A, cột 3. Ký hiệu −∞/+∞ là vô cực; "cắt tỉa" nghĩa là nút/nhánh không được thăm.

Bài 1. Bài toán 4-Queens và Hill Climbing

a) Trạng thái khởi đầu: hậu 1 ở A3, hậu 2 ở B1, hậu 3 ở B2, hậu 4 ở C2. Có 4 cặp hậu đang tấn công nhau (A3–B2, B1–B2, B2–C2, B1–C2) nên *num_conflicts* = 4. Xét mọi nước đi hợp lệ (đường đi không bị chặn) của từng hậu, số xung đột tốt nhất mà mỗi hậu có thể đạt:

Hậu	Nước đi tốt nhất	Số xung đột
Hậu 1 (A3)	không có nước nào giảm được (vd A3→A4 vẫn 4)	4
Hậu 2 (B1)	B1→D1	3
Hậu 3 (B2)	**B2→D4** (đi chéo qua C3 trống)	**1**
Hậu 4 (C2)	C2→C4	2

Láng giềng tốt nhất duy nhất là B2→D4 (1 < 4) nên thuật toán di chuyển hậu 3. **Trạng thái sau một vòng lặp: hậu 1 = A3, hậu 2 = B1, hậu 3 = D4 (đã di chuyển), hậu 4 = C2**; còn đúng 1 xung đột (B1–C2).



Hình 1a. Trạng thái trước và sau một vòng lặp

b) Thuật toán **có dừng**. Ở vòng lặp 2, các láng giềng tốt nhất (C2→D2 hoặc C2→C4) đều có 1 xung đột, không nhỏ hơn 1 của trạng thái hiện tại, nên điều kiện *neighbor.num_conflicts* ≥ *current.num_conflicts* đúng và thuật toán trả về A3, B1, C2, D4.

- ☐ A) Không dừng
- ☒ **B) Cực trị cục bộ (local optimum)** – không láng giềng nào tốt hơn (1 xung đột)
- ☐ C) Cực trị toàn cục – sai, vì nghiệm toàn cục có 0 xung đột (vd A2, B4, C1, D3)
- ☒ **D) Vai bằng (shoulder)** – có 2 láng giềng bằng giá trị (1 xung đột) và từ đó chỉ cần thêm 1 bước là đạt 0 xung đột; thuật toán dừng do dùng điều kiện ≥
- ☐ E) Không thuộc các trường hợp trên

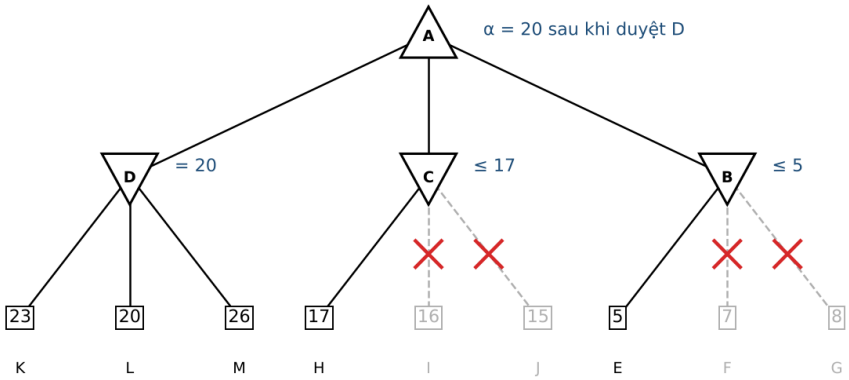
Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

a) Nút cha A là MAX nên xếp theo f giảm dần: f(D)=19 > f(C)=14 > f(B)=4.

- ☐ B → C → D
- ☐ C → D → B
- ☒ **D → C → B**

b) Duyệt theo thứ tự D, C, B (các cháu từ trái sang phải): D: K=23, L=20, M=26 → D = min = 20, nên α(A) = 20. C: lá đầu H=17 ≤ α=20 → cắt, bỏ qua I, J. B: lá đầu E=5 ≤ α=20 → cắt, bỏ qua F, G. Giá trị gốc = max(20, ≤17, ≤5) = 20, nước đi A→D.

Các nút không được thăm do bị cắt tỉa: F, G, I, J.



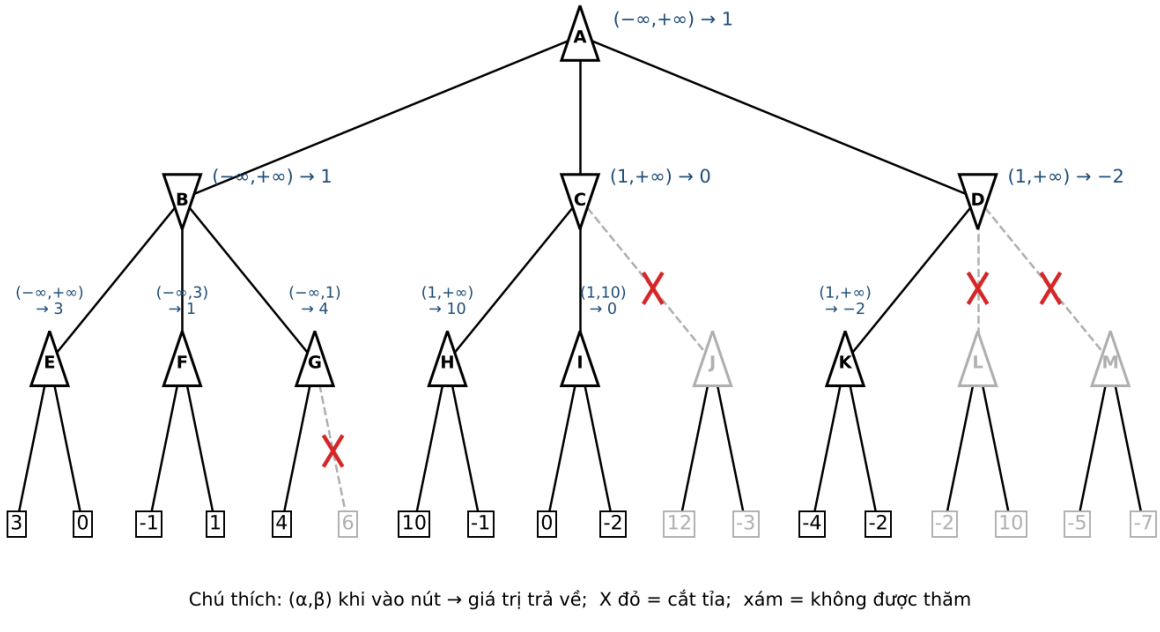
Hình 2a. Cây sau khi sắp xếp lại (D → C → B) và các nhánh bị cắt

c) ☐ Có ☒ **Không**

Alpha-Beta và Minimax cho cùng kết quả nếu chạy trên cùng một cây. Việc đổi thứ tự xét các nút con chỉ ảnh hưởng đến số nhánh được cắt tỉa, chứ không làm đổi giá trị cuối cùng. Tuy nhiên, bài này chỉ tìm đến độ sâu 2 nên thuật toán dùng giá trị f để ước lượng các trạng thái chưa kết thúc. Vì vậy, kết quả có thể khác với Minimax khi xét toàn bộ cây. Thứ tự D → C → B được chọn dựa trên các giá trị f, giúp cắt tỉa hiệu quả hơn nhưng không đảm bảo nước đi đó là tốt nhất trong toàn bộ trò chơi.

Bài 3. Adversarial Search – Alpha-Beta Pruning

Gốc A là MAX, tầng 1 (B, C, D) là MIN, tầng 2 (E…M) là MAX, mỗi nút MAX có 2 lá. Duyệt từ trái sang phải với (α, β) ban đầu = (−∞, +∞).



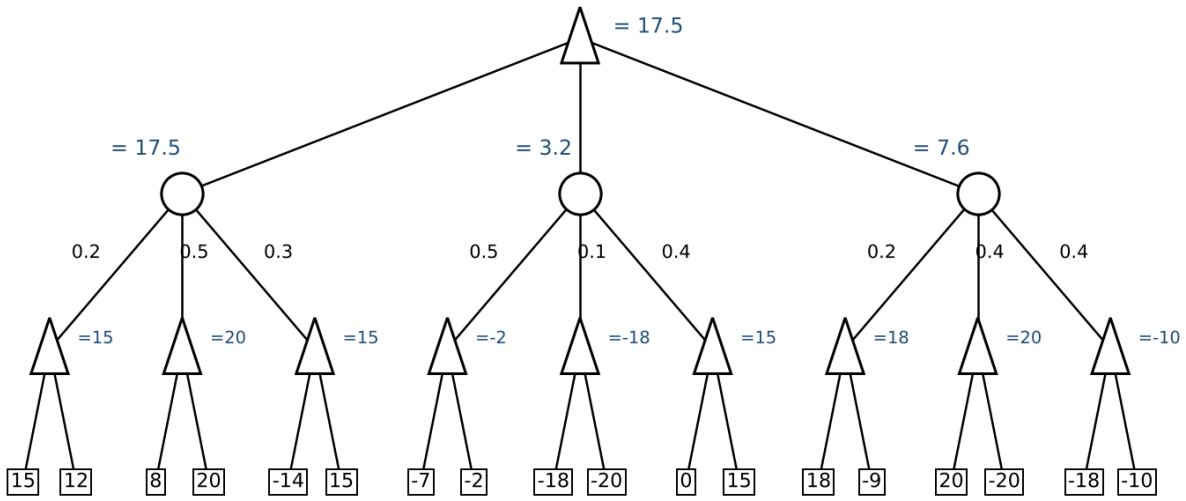
Hình 3a. Giá trị (α, β) khi vào mỗi nút, giá trị trả về và các nhánh bị cắt tỉa

Nút	Loại	(α, β) khi vào	Trả về	Ghi chú
E	MAX	(−∞, +∞)	3	lá 3, 0 → max = 3
F	MAX	(−∞, 3)	1	lá −1, 1 → 1 (1 < β=3 nên không cắt)
G	MAX	(−∞, 1)	4	lá 4 ≥ β=1 → **cắt β**, bỏ lá 6
B	MIN	(−∞, +∞)	1	min(3, 1, 4) = 1 → α(A) = 1
H	MAX	(1, +∞)	10	lá 10, −1 → 10
I	MAX	(1, 10)	0	lá 0, −2 → 0
J	MAX	—	—	**cắt α**: C đã có v=0 ≤ α=1 → bỏ J (lá 12, −3)
C	MIN	(1, +∞)	0	trả về 0 (cận trên, ≤ α=1 nên A không chọn)
K	MAX	(1, +∞)	−2	lá −4, −2 → −2
L, M	MAX	—	—	**cắt α**: D có v=−2 ≤ α=1 → bỏ L, M (lá −2, 10, −5, −7)
D	MIN	(1, +∞)	−2	trả về −2 (cận trên)
A	MAX	(−∞, +∞)	**1**	max(1, 0, −2) = 1

Kết quả: giá trị Minimax tại gốc A = **1**; nước đi tối ưu của MAX là đi tới nút **B** (nhánh trái). Các nhánh bị cắt (X): lá 6 dưới G, nút J (lá 12, −3), nút L và M (lá −2, 10, −5, −7) – tổng 7/18 lá không được thăm.

Bài 4. Expectimax và Chance Nodes

a) Mỗi nút MAX lấy lá lớn nhất; mỗi chance node lấy kỳ vọng theo xác suất trên cạnh.



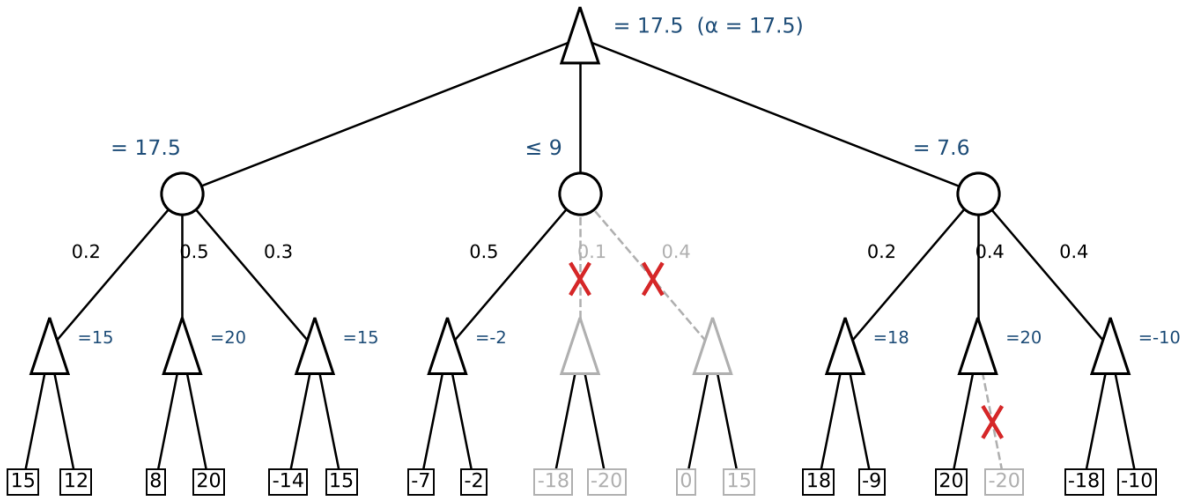
Hình 4a. Giá trị tại từng nút (Expectimax)

Chance node	Giá trị 3 nút MAX con	Kỳ vọng
Trái (0.2, 0.5, 0.3)	max(15,12)=15; max(8,20)=20; max(−14,15)=15	0.2·15 + 0.5·20 + 0.3·15 = 3 + 10 + 4.5 = **17.5**
Giữa (0.5, 0.1, 0.4)	max(−7,−2)=−2; max(−18,−20)=−18; max(0,15)=15	0.5·(−2) + 0.1·(−18) + 0.4·15 = −1 − 1.8 + 6 = **3.2**
Phải (0.2, 0.4, 0.4)	max(18,−9)=18; max(20,−20)=20; max(−18,−10)=−10	0.2·18 + 0.4·20 + 0.4·(−10) = 3.6 + 8 − 4 = **7.6**

Nút gốc (MAX) = max(17.5, 3.2, 7.6) = **17.5**, chọn nhánh trái.

b) Với dữ liệu hiện tại, không thể cắt tỉa nhánh nào mà vẫn chắc chắn giữ nguyên kết quả. Giá trị của chance node được tính từ tất cả các nhánh theo xác suất tương ứng. Nếu chưa biết giá trị của một lá, ta cũng chưa biết nó có thể làm kỳ vọng tăng hoặc giảm bao nhiêu. Do đề bài không giới hạn giá trị của các lá, ta không có cận trên hoặc cận dưới để kết luận rằng một nhánh không thể ảnh hưởng đến kết quả. Vì vậy, cần duyệt đủ cả 18 lá.

c) Với lá ∈ [−20, 20], ta có cận trên của một chance node = (tổng p·giá trị đã biết) + (xác suất còn lại)·20. Nếu cận này ≤ giá trị tốt nhất đã có ở gốc thì cắt phần còn lại.



Hình 4c. Cây Expectimax sau khi cắt tỉa với lá ∈ [−20, 20]

• **Chance trái:** chưa có ngưỡng nào để so sánh nên duyệt đủ → 17.5, gốc có α = 17.5.
• **Chance giữa:** sau nhánh 1 (MAX = −2, p = 0.5), cận trên = 0.5·(−2) + (0.1 + 0.4)·20 = 9 < 17.5 → **cắt hai nhánh còn lại** (các lá −18, −20, 0, 15).
• **Chance phải:** nhánh 1 = 18, cận trên = 0.2·18 + 0.8·20 = 19.6 > 17.5 nên tiếp tục. Nhánh 2: lá đầu 20 đã là giá trị lớn nhất có thể nên MAX = 20 và **cắt lá −20**. Nhánh 3 = −10. Giá trị = 3.6 + 8 − 4 = 7.6 < 17.5.
Kết quả: giá trị gốc = 17.5, nước đi tối ưu là nhánh trái. Có 5/18 lá không được thăm (−18, −20, 0, 15 ở chance giữa và −20 ở chance phải).

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking (FC) khác AC-3 như thế nào?

Tiêu chí	Forward Checking	AC-3
Phạm vi lan truyền	Chỉ một bước: sau khi gán X = v, chỉ loại khỏi miền của các biến *chưa gán kề X* những giá trị xung đột với v. Không lan truyền tiếp.	Lan truyền dây chuyền: khi miền của một biến bị thu hẹp thì đưa lại các cung liên quan vào hàng đợi, lặp đến khi không còn thay đổi; ảnh hưởng đến cả các biến không kề biến vừa gán.
Thời điểm thực hiện	Trong quá trình tìm kiếm quay lui, sau *mỗi phép gán*.	Thường là tiền xử lý trước khi tìm kiếm; cũng có thể gọi sau mỗi phép gán (MAC).
Chi phí tính toán	Thấp: mỗi phép gán chỉ kiểm tra các biến chưa gán kề biến đó, cỡ O(số hàng xóm · d).	Cao hơn: tối đa $O(e \cdot d^3)$ với e cung và d là kích thước miền.
Khả năng phát hiện thất bại	Phát hiện miền rỗng của hàng xóm trực tiếp; bỏ sót xung đột gián tiếp.	Mạnh hơn (đảm bảo nhất quán cung), phát hiện được dead end sớm hơn FC.

Nhìn chung, FC thực hiện ít kiểm tra hơn sau mỗi lần gán nên chi phí thấp, nhưng có thể không phát hiện được các xung đột gián tiếp. AC-3 kiểm tra sự nhất quán giữa các miền rộng hơn nên thường tốn công hơn, đổi lại có thể phát hiện bế tắc sớm hơn.

b) Vì sao dùng các heuristic sau trong CSP?

i) Minimum Remaining Values (MRV) chọn biến còn ít giá trị hợp lệ nhất. Ý tưởng là xử lý biến khó trước: nếu biến đó không còn giá trị nào phù hợp, ta sẽ phát hiện nhánh không có nghiệm sớm thay vì tiếp tục gán các biến khác. Nếu biến chỉ còn một giá trị thì ta cũng có thể gán ngay giá trị đó.

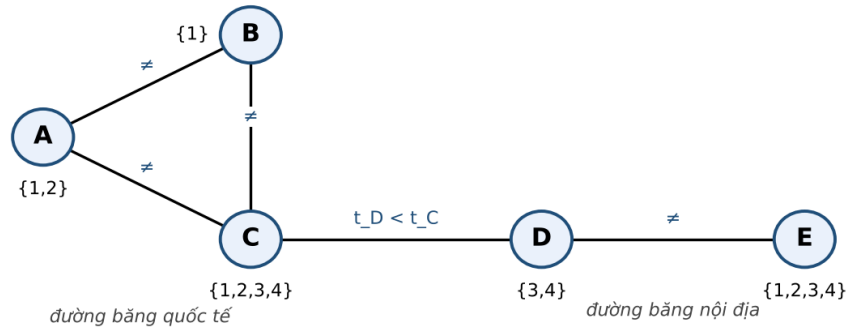
ii) Least Constraining Value (LCV) dùng sau khi đã chọn biến: ta thử giá trị làm mất ít lựa chọn nhất của các biến liên quan. Nhờ vậy, những biến còn lại vẫn có nhiều khả năng để gán hơn. Có thể hiểu đơn giản là MRV quyết định nên xét biến nào trước, còn LCV quyết định nên thử giá trị nào trước.

Bài 6. CSP – Điều phối không lưu

a) Mô hình hóa. Biến: A, B, C, D, E (mỗi máy bay một biến). Gọi giá trị của biến X là cặp (t_X, r_X), với khung giờ $t \in \{1, 2, 3, 4\}$ và đường băng $r \in$ {QT (quốc tế), ND (nội địa)}. Miền của mỗi biến: {1,2,3,4} × {QT, ND} (8 giá trị).

Ràng buộc đơn (unary): $t_B = 1$; $t_D \geq 3$; $t_A \leq 2$.
Ràng buộc nhị phân (binary): $t_D < t_C$ (D hạ cánh trước khi C cất cánh); và với mọi cặp $X \neq Y$: $\neg(t_X = t_Y \wedge r_X = r_Y)$ (không hai máy bay nào cùng khung giờ trên cùng đường băng).

b) Với hai ràng buộc bổ sung (A, B, C chỉ dùng đường băng quốc tế; D, E chỉ dùng đường băng nội địa), đường băng đã được xác định bởi biến nên miền mới chỉ là khung giờ: $D_A = D_B = D_C = D_D = D_E = \{1, 2, 3, 4\}$. Ràng buộc còn lại: unary $t_B = 1$, $t_D \geq 3$, $t_A \leq 2$; binary $A \neq B$, $A \neq C$, $B \neq C$ (cùng đường băng quốc tế), $D \neq E$ (cùng đường băng nội địa), $t_D < t_C$. Các cặp khác đường băng (vd A và D) không còn ràng buộc khác nhau. Đồ thị ràng buộc (miền hiển thị sau khi áp dụng ràng buộc đơn):



Hình 6b. Đồ thị ràng buộc: tam giác A–B–C, cung C–D và cung D–E

c) Arc consistency. Áp dụng ràng buộc đơn trước, sau đó AC-3. Giá trị bị loại do ràng buộc đơn được gạch xám, bị loại do AC-3 được gạch đỏ:

Biến	Miền (1 2 3 4)	Miền còn lại	Lý do loại bỏ
A	~~1~~ **2** ~~3~~ 4	{2}	3, 4: $t_A \leq 2$ (unary); 1: B = 1 nên A ≠ 1

B	**1** ~~2~~ ~~3~~ ~~4~~	{1}	2, 3, 4: $t_B = 1$ (unary)
C	~~1~~ ~~2~~ ~~3~~ **4**	{4}	1: B = 1; 2: A = 2; 3: cần $t_D < 3$ nhưng D ∈ {3, 4}
D	~~1~~ ~~2~~ **3** ~~4~~	{3}	1, 2: $t_D \geq 3$ (unary); 4: không có giá trị C > 4
E	**1** **2** ~~3~~ **4**	{1, 2, 4}	3: D = 3 nên E ≠ 3

Thứ tự lan truyền: (A,B) → A{2}; (C,B) → C{2,3,4}; (C,A) → C{3,4}; (D,C) → D{3}; (C,D) → C{4}; (E,D) → E{1,2,4}. Kết quả: **A = 2, B = 1, C = 4, D = 3, E ∈ {1, 2, 4}**.

d) Forward Checking + Backtracking (MRV chọn biến; hòa → ký tự nhỏ hơn; chọn giá trị nhỏ hơn trước; đã áp dụng ràng buộc đơn). Miền ban đầu: A{1,2}, B{1}, C{1,2,3,4}, D{3,4}, E{1,2,3,4}.

Bước	Biến chọn (MRV)	Gán	Miền sau Forward Checking	Kết quả
1	B (1 giá trị)	B = 1	A{2}, C{2,3,4}	tiếp tục
2	A (1 giá trị)	A = 2	C{3,4}	tiếp tục
3	C và D cùng 2 giá trị → C	**C = 3**	D{ } (cần $t_D < 3$ nhưng D ≥ 3)	**dead end** → hoàn tác C = 3
4	C	C = 4	D{3}	tiếp tục
5	D (1 giá trị)	D = 3	E{1,2,4}	tiếp tục
6	E	E = 1	—	đủ 5 biến → nghiệm

Các cặp (biến, giá trị) theo thứ tự phát sinh: (B,1), (A,2), (C,3) – bị hoàn tác, (C,4), (D,3), (E,1). **Nghiệm:** A = khung 2, B = khung 1, C = khung 4 (đều đường băng quốc tế); D = khung 3, E = khung 1 (đều đường băng nội địa).

Bài 7. CSP – Magic Square

a) Mô hình hóa. Biến: A, B, C, D, E (các ô chưa biết). Miền: {1, …, 9} (thực chất {1, 3, 7, 8, 9} vì 2, 4, 5, 6 đã dùng). Ràng buộc:

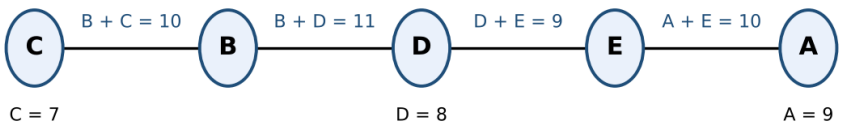
• AllDiff(A, B, C, D, E) và khác các giá trị đã cho 2, 4, 5, 6.

• Hàng 1: 4 + A + 2 = 15 (unary: A = 9). Hàng 2: B + 5 + C = 15 (B + C = 10). Hàng 3: D + E + 6 = 15 (D + E = 9).

• Cột 1: 4 + B + D = 15 (B + D = 11). Cột 2: A + 5 + E = 15 (A + E = 10). Cột 3: 2 + C + 6 = 15 (unary: C = 7).

• Đường chéo 4 + 5 + 6 = 15 luôn đúng; đường chéo kia: 2 + 5 + D = 15 (unary: D = 8).

b) Đồ thị ràng buộc nhị phân (bỏ qua AllDiff): bốn ràng buộc nhị phân tạo thành một đường C – B – D – E – A.



Hình 7b. Đồ thị ràng buộc nhị phân (ghi kèm ràng buộc đơn)

c) Thu hẹp miền bằng AC-3. Ràng buộc đơn: A = 9, C = 7, D = 8; B và E bắt đầu với miền {1, …, 9}.

Cung xét	Ràng buộc	Miền sau khi xét
(B, C)	B + C = 10, C = 7	B{3} (loại 1, 2, 4–9)
(B, D)	B + D = 11, D = 8	B{3} vẫn nhất quán
(E, D)	D + E = 9, D = 8	E{1} (loại 2–9)
(E, A)	A + E = 10, A = 9	E{1} vẫn nhất quán
(C,B), (D,B), (D,E), (A,E)	xét ngược lại	không còn giá trị nào bị loại

Tất cả miền đều còn đúng một giá trị nên tìm được nghiệm ngay mà không cần quay lui: **A = 9, B = 3, C = 7, D = 8, E = 1**.

4	**9**	2
3	5	**7**
8	**1**	6

Kiểm tra: mọi hàng, cột và hai đường chéo đều có tổng 15; chín số 1…9 đều khác nhau.

Bài 8. MRV và LCV trong thực hành

Giả thiết: ba người phải chọn màu đôi một khác nhau; dùng Forward Checking để cập nhật miền; thứ tự cầu vồng khi hòa giá trị: Đỏ < Vàng < Xanh lá < Hồng; khi hòa biến thì chọn theo thứ tự chữ cái (AJ, Ellyse, Sunny). Miền ban đầu: AJ{Đỏ, Vàng, Xanh lá}, Sunny{Đỏ, Vàng, Hồng}, Ellyse{Đỏ, Hồng}.

a) MRV + LCV

Bước	Chọn biến (MRV)	Chọn giá trị (LCV)	Miền sau FC
1	Ellyse (2 giá trị, ít nhất)	Đỏ loại 2 giá trị (khỏi AJ, Sunny); Hồng loại 1 (khỏi Sunny) → **Hồng**	AJ{Đỏ,Vàng,Xanh lá}, Sunny{Đỏ,Vàng}
2	Sunny (2) < AJ (3)	Đỏ loại 1, Vàng loại 1 → hòa, theo cầu vồng chọn **Đỏ**	AJ{Vàng,Xanh lá}
3	AJ	Vàng và Xanh lá đều loại 0 → hòa, chọn **Vàng**	—

Thứ tự gán: Ellyse = Hồng → Sunny = Đỏ → AJ = Vàng. **Số lần backtrack: 0.**

b) Maximum Remaining Values + Most Constraining Value

Bước	Chọn biến (Max remaining)	Chọn giá trị (Most constraining)	Kết quả
1	AJ và Sunny cùng 3 giá trị → AJ	Đỏ loại 2 (khỏi Sunny, Ellyse), Vàng loại 1, Xanh lá loại 0 → **Đỏ**	Sunny{Vàng,Hồng}, Ellyse{Hồng}
2	Sunny (2) > Ellyse (1)	Hồng loại 1 (khỏi Ellyse), Vàng loại 0 → **Hồng**	**Ellyse{ } → dead end, backtrack lần 1**
3	Sunny	giá trị còn lại: **Vàng**	Ellyse{Hồng}
4	Ellyse	**Hồng**	nghiệm

Nghiệm: AJ = Đỏ, Sunny = Vàng, Ellyse = Hồng. **Số lần backtrack: 1**, nhiều hơn câu (a) (0 lần).

Giải thích: Ở câu (a), MRV chọn Ellyse vì chỉ còn hai màu để chọn, còn LCV ưu tiên màu Hồng vì màu này ảnh hưởng ít hơn đến lựa chọn của hai người còn lại. Nhờ cách chọn đó, ta tìm được nghiệm mà không phải quay lui. Ở câu (b), cách chọn ngược lại khiến AJ chọn Đỏ trước, sau đó Sunny chọn Hồng và Ellyse không còn màu phù hợp. Vì vậy, thuật toán phải quay lui rồi thử giá trị khác. Với ví dụ này, số lần quay lui chỉ chênh nhau một lần, nhưng nó cho thấy thứ tự chọn biến và giá trị có thể ảnh hưởng đến quá trình tìm nghiệm.